Kayla Ryan
05/04/2026

Final Project Report

1. Introduction

The primary objective of this project is to create a supervised machine learning model that can accurately predict the length of stay for traditional domestic pets (specifically cats and dogs) entering a municipal shelter. This is a multi-class classification problem where the subject is classified into one of four categories: nominal, elevated, severe, and critical. These categories represent the subject's risk of having a long stay at the shelter. These are distinct tiers: with nominal risk represents stays up to a week, elevated risk represents stays from a week up to a month, severe risk represents stays from one to three months in length, and the critical risk highlights the most important cases where residents stay in the shelter system for three months or longer, consuming the most resources on an individual basis. By determining the length of stay for an animal at the point of intake, this project aims to provide a prescriptive tool for shelter management to optimize the facility's capacity and animal flow by allocating resources to the animals that would most benefit.

The motivation to solve this problem is to improve animal welfare through the increased operational efficiency of municipal shelters. According to official operational dashboards (Austin Animal Center, n.d.) and the Austin Animal Advisory Commission records the shelter regularly operates at or above capacity, especially in the category of medium and large dogs (Hayden-Howard, 2024), meaning the number of animals housed at the facility often exceeds the number of appropriate, permanent kennels available. This is displayed on the Shelter Capacity Dashboard as a negative number. Operating beyond maximum capacity can easily force the use of temporary crates in spaces like conference rooms, hallways, and offices (Bland, 2023), increasing animal stress and the potential for the spread of disease.

In a "no-kill" shelter where the live release goal is 95% or better, the only way to ease the negative capacity is to reduce the average number of days an animal spends in one of their kennels. By using a tool (like the one created in this project) to identify those "critical risk" animals at intake, shelter staff are able

to bypass the traditional “wait-and-see” period and instead take immediate action to find foster placement or begin targeted adoption marketing on social media and similar. The ability to predict long-term residents would transform the shelter’s strategy from reactive crisis management to proactive resource allocation, which would directly impact the facility’s ability to return to a sustainable capacity for care.

2. Data Exploration and Preprocessing

Exploration and Cleaning

The data pulled from the Austin Animal Center website consisted of two CSV files. After creating the project notebook, I checked the names of the features in each dataframe as well as the data type for each. I determined that all features across both dataframes were stored as string objects which would require significant preprocessing. I checked the first few rows of each dataframe to get an idea of what kind of values I could expect for each feature before moving on.

As the goal of the project is determining an animal’s length of stay prior to adoption, I needed to remove other outcomes like euthanasia, disposal, and transfer. Additionally, I removed reunions including “Return to Owner” and “RTO-Adopt”, as these are not true adoptions based on the subject’s independent features, and this could cause target bias.

Examining a sample of the values in the “Found Location” column, I quickly determined that this feature would need to be removed as there was too much noise and human error (such as typos like “Norht”) to justify the extensive preprocessing necessary to find order in these values.

Preprocessing

In standardizing the data, I converted the intake and outcome the timestamps from strings to a standard DateTime format. I split the “MonthYear” feature into two integer features representing both month and year. The Age at intake was a bit tricky as it varied from a few days old, to a couple months, to many years old, so this was converted into a uniform unit of days as an integer.

Looking at the species in the intake dataframe, there were much more than just cats and dogs listed, but since they're not the focus of the project and those other animals don't follow the same standard adoption timelines and processes, they were removed.

The values of the remaining features in the intake table were evaluated and consolidated before determining which values are significant values before being encoded, with the most frequent value of each original feature being dropped and used as the baseline value for that feature. The name feature was simplified so that rather than encoding the subject's name, I created a binary variable representing whether they had a name at intake, whether existing or given by staff immediately. The reproductive status needed to be broken apart before encoding as it was a compound value including the sex as well as whether the animal had been 'fixed'.

Because cats have breeds and color patterns that are exclusive to them and dogs are the same way, I pulled the significant values using the 1% frequency threshold species separately to ensure they were equally represented. Again, the baseline values of each were dropped after encoding.

After making sure that there were no null values in the intake dataframe, I merged the two tables with an inner join on the animal ID. Because the outcome table had already been narrowed to only those with an adoption outcome, only those animal IDs would be matched with the animals in the intake table. The unmatched records were dropped. I then created a temporary column to determine the length of stay for each animal by subtracting the intake timestamp from the outcome timestamp. This was a little complicated because of those animals with multiple stays. If an animal had two separate stays, they had two intake timestamps and two outcome timestamps. The system initially matched all possibilities, resulting in 4 stays, including one where the animal traveled back in time and was adopted before it was put into the system. Finding these and removing them was straightforward, but I also had to find and remove the other erroneous stays. For this, I sorted first by Animal ID, then by length of stay. This way, the earliest intake timestamp with the shortest stay was first, followed by an intake at the same timestamp

but with a much longer stay, then by an intake timestamp much later. This allowed me to confidently remove the duplicate entries and leave the two legitimate stays.

After this, I created and defined the categorical target risks and dropped unnecessary features prior to defining the final features and target tables.

3. Modeling

To build the most reliable models, I used a pipeline with a 5-fold stratified k-fold cross validation. Because “critical risk” animals only make up a small portion of the dataset, stratification was necessary to make sure that every training and testing fold had a balanced representation of those high-risk cases. I used Chi Squared testing for feature selection, applied standard scaling to normalize the data, and ran a GridSearch to tune the parameters for each model I tested.

I ran five different models, starting with the Logistic Regression as a baseline. I moved to the Decision Tree to see if it would find better patterns by splitting the data into ‘if then’ branches. I decided to take another step in that direction with the Random Forests model to see if many trees would perform better than one. Based on the results of the Decision Tree and Random Forests, I wanted to take it just one step further and try the Histogram-Based Gradient Boosting Classifier (HGBC) that I was introduced to this semester in a data science course, to see if the way it handles tabular data would mean outperforming the others. Finally, I implemented the Artificial Neural Network (ANN) model, using a sequential structure with a 10-node hidden layer and a 4-node output layer to generate the probabilities for each risk tier.

To handle the class imbalance, I used custom class weights where the larger groups of “Nominal” and “Elevated” were weighted as 1, the smaller “Severe” was a 1.5 and the smallest and most important tier “Critical” was weighted as 5.0. This was to tell the models that missing a long stay was 5 times more costly than misclassifying a short-term animal.

When evaluating the models, I prioritized Recall for the "Critical" class over others. In a shelter, the most important goal is to flag the long stays as soon as possible. A false positive is a minor issue of unnecessarily spending some resources, but a false negative leads to overcrowding and additional stress on the animal.

I believe the final HGBC model is a good fit. The gap between the training accuracy (48.52%) and the testing accuracy (46.23%) is only 2.29%, showing that the model is finding patterns well and not just memorizing the training data. Catching 67% of the highest risk animals on day 1 proves that this is a major win and can help shelter management to shift from a reactive response to a proactive one.

4. References and Citations

Austin Animal Center. (n.d.) *Shelter capacity and capacity for care.* City of Austin. https://www.austintexas.gov/animal-services/shelter-capacity

Bland, D. (2023, June 26). *Response to Council Member Kelly's concerns at the Austin Animal Center [Memorandum]*. City of Austin. https://services.austintexas.gov/edims/document.cfm?id=410686

Hayden-Howard, S. (2024, May 24). *Austin Animal Center at critical capacity [Memorandum]*. City of Austin. https://services.austintexas.gov/edims/document.cfm?id=429759

GeeksforGeeks. (2024, June 6). *HistGradientBoostingClassifier in Sklearn*. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/histgradientboostingclassifier-in-sklearn/

GeeksforGeeks. (2025, August 23). *Multiclass logistic regression*. GeeksforGeeks. https://www.geeksforgeeks.org/artificial-intelligence/multiclass-logistic-regression/

GeeksforGeeks. (2020, July 7). *How to Join Pandas DataFrames using Merge?* GeeksforGeeks. https://www.geeksforgeeks.org/python/how-to-join-pandas-dataframes-using-merge/

Scikit-Learn. (2019). *User guide: contents — scikit-learn 0.22.1 documentation*. Scikit-Learn.org. https://scikit-learn.org/stable/user_guide.html

Scikit-learn. (2019). *3.2.4.3.1. sklearn.ensemble.RandomForestClassifier — scikit-learn 0.21.3 documentation*. Scikit-Learn.org. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html#sklearn.ensemble.RandomForestClassifier

sklearn.ensemble.HistGradientBoostingClassifier. (n.d.). Scikit-Learn. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingClassifier.html

Team, K. (n.d.). *Keras documentation: The Functional API*. Keras.io. https://keras.io/guides/functional_api/

Team, K. (n.d.). *Keras documentation: Developer guides*. Keras.io. https://keras.io/guides/